

INTERNSHIP PROGRESS REPORT

ADS Softek

Chapterwise

Textbook PDF to Ordered Topics, Explanations, Grounded Q&A, Images and TTS

Company	ADS Softek
Intern	Dara Eakeswar Rayudu
Role	AI Intern
First working day on the project	Thursday, 23 July 2026
Last working day on the project	Wednesday, 2 September 2026
Project	Chapterwise (full-stack educational web application)
Reporting period	23 July 2026 – 2 September 2026
Duration	Six calendar weeks (through last working day, 2 September 2026)
Primary stack	Python / FastAPI, React (Vite), PyMuPDF, RapidOCR, Azure OpenAI
Optional infra	Kaggle GPU notebook (Llama 3.2 1B) via Cloudflare tunnel
Reference textbook	NCERT Class 9 Science, about 301 MB, 1,299 pages

Prepared from project development sessions, git history, and the living project changelog. Last development session: 2 September 2026. Credentials and API keys are omitted.

1. Executive summary

Chapterwise converts a school textbook PDF into a document-order topic tree (chapter to section to subsection). Each topic presents a grounded explanation, source page range, diagrams or photographs, topic-scoped question answering, and text-to-speech narration. Unlike a magazine-style layout product, topics remain hierarchical and follow the book, not font size or page design.

Work was delivered in numbered product steps, with a test-and-approve loop after each major piece: extract, topic tree, user interface, explanations, Q&A, TTS, image handling, then polish.

Over six weeks the application moved from an empty monorepo to a working end-to-end system that can ingest the NCERT Class 9 Science textbook locally, build on the order of 223 topics in under a

second on the regex path, stream explanations and answers, classify and enhance images, and narrate with a selectable Azure TTS voice.

2. Approach

2.1 Product constraints

- Local-first extraction. Do not send the full PDF to a cloud model for parsing in v1. Use PyMuPDF and OCR on CPU; use language models only for language tasks (tree, explanation, Q&A).
- Ordered hierarchy. The tree must follow textbook headings, not visual layout.
- Grounded generation. Explanations and answers must stay inside the open topic’s source; if the section does not contain the answer, the model must say so.
- Cost and safety for images. Formulae and labelled diagrams must never be regenerated by an image model. Decorative photos may be enhanced on demand only.
- Fail soft. A dead tunnel or Azure error must not freeze the UI; show source text and a clear status.

2.2 Engineering method

- Follow a published eight-step build order (upload/extract, topics, frontend tree, topic view, ask, TTS, image enhance, polish).
- Implement one step, then test (often on pages 1–8, then on harder pages, then the full book).
- Measure before changing (extract timing phases, GPU out-of-memory logs, Cloudflare 502/524, classification scores).
- Prefer a simple reliable path when a fancy path is unstable (regex table of contents versus a 1-billion-parameter Llama JSON tree).
- Keep a living project log so the next day starts from facts, not memory.

2.3 Architecture as shipped

Layer	Role
Backend FastAPI (port 8766)	Upload, extract, topic tree, topic payload, SSE explanation/Q&A, TTS, image generate
Frontend Vite React (port 5173)	Upload, hierarchical sidebar, topic view, Ask drawer, Listen and voice picker
Azure (split resources)	Chat on a dedicated text resource; TTS and gpt-image-2 on a services endpoint
Optional Kaggle	Llama chat and/or remote extract behind a Cloudflare tunnel

3. Challenges, root causes, and solutions

The following table summarises the main engineering problems encountered, what actually failed, and the fix that was verified in testing.

Challenge	What went wrong	How it was solved
300 MB textbooks felt stuck on upload	The API waited for full extraction of every page, not just the HTTP upload. Header/footer analysis originally scanned all 1,299 pages even for a 1–10 page test (about 93 s of about 99 s).	Page-range extract; sample 36 pages for headers and cache by file fingerprint; parallel workers; later defer image base64. Partial extract dropped to about 6–8 seconds.
Kaggle + Cloudflare timeouts (502 / 524 / SSL EOF)	Synchronous upload-and-extract and chat completions exceeded free-tunnel limits (about 100 seconds).	Chunked 4 MB upload; async extract/chat plus poll; treat 502/503/524 as tunnel blips; 409 with job id when busy; fail fast if DNS is dead so opening a topic still returns source text.
Azure DeploymentNotFound for chat	TTS (gpt-4o-mini-tts) and images (gpt-image-2) worked; chat was never deployed on that resource. Keys and URLs were mixed.	Split clients: dedicated chat credentials versus TTS/image keys. Later a student Azure chat resource (gpt-4.1-mini) became the default text provider.
Llama 1B as topic builder	Malformed JSON, copied few-shot titles, invented subsections, CUDA out of memory (about 13 GB attention spike), hangs on ~12k-character prompts.	JSON repair and retry; tighter prompts; repetition penalty; token caps; regex heading TOC as default (full book: 223 topics in 0.16 s). LLM path kept behind a flag.
Gated Hugging Face and Kaggle secrets	401 on Llama: missing HF token secret versus token present but no model access versus mismatched Kaggle API secret.	Load secrets via Kaggle User Secrets; print present/length only; Hugging Face login; accept the gated model licence.
Dead Kaggle blocked opening a topic	Client retried connection errors until a 600-second LLM timeout.	After inner retries, raise unreachable immediately; UI shows explanation failed with source in about 3 seconds.
Full-book extract memory and time	Eager base64 of every image; RapidOCR constructed once per worker thread. First full run about 287 s, peak about 2.4 GB.	Default defer image encoding; single RapidOCR under a lock. Later full run about 209 s. Topic endpoint encodes only that topic's images.
Black gallery images	1-bit ICC Gray plus soft mask: raw PNG looked solid black (artwork lived in the mask).	Pixmap decode to RGB PNG; mask as alpha; empty colour plus mask becomes a transparent PNG.
HD generate invented scenes	Text-only image generation plus chapter explanation in the prompt turned wheat grains into a farm. Colour diagrams were labelled decorative because PDF text overlap was 0.	On-demand Generate HD only; caption-primary prompts; then images.edit with source bytes and high input fidelity. RapidOCR on the crop: baked labels become informational (Lanczos only). Classify in the background so topic

		open stays fast.
Streaming versus classify poll	Polling the topic endpoint every 3 s returned a null explanation and wiped in-progress streamed text.	Merge poll payloads: keep streaming text; cache explanation only when the stream completes.
False Chapter N headings	Prose such as “see Chapter 7” became extra chapter nodes (dozens of duplicates / out of order).	Tighten chapter regex to start-of-line headings. Layout/OCR reading order remains a known limit; a document-layout model was investigated, not adopted as default.

4. Improvements delivered

4.1 Performance

- Header/footer sampling plus fingerprint cache.
- Parallel extract (thread pool on Windows).
- Deferred image base64; on-demand encode plus in-memory cache.
- Regex table of contents: full book in 0.16 s versus a multi-minute or failing Llama tree.

4.2 Reliability

- Async Kaggle jobs, retries, chat watchdog, chat reset.
- Split Azure credentials; dedicated debug endpoints for text and Azure services.
- Fail-soft explanations, Q&A, and HD generate.

4.3 Correctness

- Body text is not dropped: each heading’s page range runs until the next heading.
- Fallback chapter Pages start–end if a range has no headings.
- Q&A grounded in explanation plus source of that topic only.
- Informational versus decorative routing; OCR gate for painted labels.
- Image edit keeps identity of photos (costume, grain pile) instead of inventing a new scene.

4.4 User experience

- Animated upload screen with elapsed time and extract/build timings.
- Sidebar inner-scroll; source card collapsed by default.
- Markdown when complete; cursor while streaming.
- Ask: floating button plus overlay drawer with a chat thread.
- TTS voice picker (10 confirmed voices), default shimmer, persists across topic switches.

5. Week-by-week work log

Weeks are calendar weeks from the first implementation day (Thursday 23 July 2026) through the last working day, Wednesday 2 September 2026. Dates below are working days evidenced in project chats and git history. There is no Week 7: no development chats exist after 2 September.

Week 1 — 23–29 July 2026

Focus: Scaffold the product, prove local extract, confront large-PDF and cloud-chat blockers.

Days	What I did
23 Jul	Step 1: FastAPI upload and pdf_extract (PyMuPDF + pdfplumber), security and size limits. Step 2: build_topics and Azure-oriented topic builder. Step 3: Vite React upload panel and topic tree. Set PDF cap to 500 MB. Configured Azure environment. Diagnosed that slowness was extract, not OpenAI. Enabled all CPU cores. Wrote the project plan file.
24 Jul	Designed Kaggle remote extract: dataset bundle, notebook GPU server, laptop client. Hit SSL EOF and Cloudflare 524 on large uploads. Added chunked upload and async extract plus poll. Confirmed Azure chat DeploymentNotFound while exploring Foundry.
25 Jul	Clarified for management which chat deployment was missing. Re-checked TTS and image URLs on the services endpoint.
27 Jul	Verified Step 3 UI. Smoke-tested TTS and image generation (working). Built Step 4 shell: topic endpoint, source and images, skeletons. Added animated upload loading. Documented that chat still had no deployment.
28 Jul	Re-audited Azure: TTS/images OK; chat still missing. Added Azure debug endpoint and upload timing in API and UI. Alternate TTS voice sample.

Outcome: Upload, extract, and sidebar tree work locally. TTS and images work. Topic LLM tree blocked on Azure chat. Kaggle extract path exists but tunnels are fragile.

Week 2 — 30 July – 5 August 2026

Focus: Unblock text generation without a chat deployment; first Llama integration; RapidOCR decision.

Days	What I did
30 Jul	Implemented Kaggle Llama 3.2 1B Instruct (4-bit) as the Kaggle text provider. Laptop text_provider routing. Debug text endpoint. Published the chapterwise-models dataset.
3 Aug	End-to-end tests through the tunnel. Fixed JSON being passed to the byte-upload helper. Removed a dead

	OpenAI client call. Added JSON repair and retry for Llama topic JSON. Granted Hugging Face gated access. First successful debug-text after model preload. Streamed LLM explanations into topic view. Hit Cloudflare 502 on large topic prompts.
4 Aug	Evaluated RapidOCR versus pdfplumber for Class 9 Science. Decided to implement RapidOCR as the laptop text extractor.

Outcome: Explanations can run on Kaggle Llama. Topic JSON from the 1B model is unreliable. RapidOCR chosen to improve extract quality and speed on mixed PDFs.

Week 3 — 6–12 August 2026

Focus: Extract performance, Llama out-of-memory, chat reliability, regex table of contents (Plan B).

Days	What I did
10 Aug	Implemented RapidOCR path (digital pages still use native get_text). Instrumented debug timing. Proved whole-PDF header scan caused about 93 s waste; cached a 36-page sample. Tightened topic prompts. Added repetition controls. Diagnosed CUDA OOM on retry; capped tokens and truncated all messages. Republished the Kaggle bundle.
11 Aug	Debugged Hugging Face token User Secret (empty versus present). Fixed 401 Invalid API token by matching the Kaggle API secret. Replaced sync chat with async chat plus poll to avoid Cloudflare 524.
12 Aug	Plan A: 409 when busy, job watchdog, generate time limit, chat reset. Topic-sized prompts still hung. Plan B: default regex topic builder — NCERT-style Chapter N, N.N, N.N.N, Activity N.N (including OCR stutter). Body kept via page spans; empty-range fallback chapter. Pages 1–8: about 21 topics in about 0.01 s. Documented OCR title spill and two-column order issues (deferred).

Outcome: Local extract is fast for page ranges. Topic tree no longer depends on Llama. Kaggle chat is more robust but is not the table-of-contents path.

Week 4 — 13–19 August 2026

Focus: Azure chat on a student resource; full-book extract; Q&A; image pipeline; Ask UX.

Days	What I did
13 Aug	Confirmed live default is regex. Dead Kaggle must not hang topic open. Added separate Azure chat key and

	base URL (must include /openai/v1). Kept extract and regex local; Azure for explanations.
14 Aug	Full-book extract: about 287 s then about 209 s after deferring base64 and locking RapidOCR init. On-demand topic image encode (JPEG2000 to PNG). Step 5: grounded Ask (smoke: Activity 1.1 Panch Tatva; off-topic boiling point refused). Full-book TOC: 223 topics / 214 headings / 0.16 s. Gallery drops background images. Default TTS voice shimmer.
17 Aug	Soft-mask black-image fix. Step 7: classify informational versus decorative; Lanczos versus generate. Do not block topic open on gpt-image-2. On-demand Generate HD; 400 if informational. Markdown rendering. Sidebar inner-scroll. Prompt fix so HD follows the photo, not the chapter story. RapidOCR on image pixels so labelled diagrams stay informational; classify pending until OCR finishes.
19 Aug	Iterated Ask UX: docked composer, then three layout experiments; shipped floating button plus overlay drawer with chat thread. Section cards, full-width topic panel, ChatGPT-style composer.

Outcome: The app works on the entire Class 9 Science book. Q&A, TTS, and a conservative image pipeline are in place. Ask sits in a drawer so reading stays full width.

Week 5 — 20–26 August 2026

Focus: Streaming, voices, source UX, faithful HD edits, image–topic alignment.

Days	What I did
20 Aug	SSE for explanations and Ask. Azure streaming; cache only on completion. Merge classify polls so OCR polls do not wipe text. Voice picker: ten confirmed voices. Source text collapsed by default. Confirmed images.edit; switched HD to edit plus high input fidelity. Smoke: tribal-dress photo and wheat pile stayed subject-faithful.
21 Aug	Health-checked backend and frontend; regression pass on streaming, voices, collapsible source, and the edits path.
26 Aug	Investigated same-page sibling topics sharing images. RapidOCR path had no word boxes, so a pdfplumber-only y-match would not help laptop extracts. Implemented line/word bounding boxes and closest-heading-above matching, in stages with coordinate checks.

Outcome: Explanations and Q&A feel interactive. HD generate is an edit of the real pixels. Image attachment starts to use geometry, not only page range.

Week 6 — 27 August – 2 September 2026

Focus: Full-book TOC quality (false chapter matches); layout-model experiment.

Days	What I did
2 Sep	Separated false Chapter N regex hits from OCR title spill, with real counts on Class 9 Science. Tightened chapter regex to line-start headings; re-counted chapter nodes. Experimented with DocLayout-YOLO (no training) on page 6 reading order and on xylem (p. 68) versus wheat (p. 684) for figure versus photo — evidence for a possible future layout path, not a production default.

Outcome: Fewer bogus chapter nodes from in-prose Chapter N mentions. Layout machine learning is an investigation, not a rewrite of v1. This was the last day of project work.

6. Current system status (as of last working day, 2 September 2026)

Step	Scope	Status
1	PDF upload and per-page extract	Complete
2	Ordered topic tree (regex default; LLM optional)	Complete for v1; remaining OCR order / leftover false chapters
3	Upload UI and topic sidebar	Complete
4	Topic view, streamed explanation, images	Complete for v1
5	Grounded Ask (SSE)	Complete
6	TTS and voice picker	Complete
7	Classify, Lanczos, on-demand HD edit	Complete for v1; overlap counts still unused on RapidOCR extract
8	End-to-end polish / v2 backlog	Partially done (UX, streaming, alignment experiments)

Typical laptop defaults: local extract, RapidOCR (digital shortcut on), regex TOC, Azure chat (gpt-4.1-mini), deferred images, shimmer TTS, on-demand images.edit.

7. What remains

- Further TOC cleanup (title spill, two-column OCR order, residual duplicate chapters from repeated lesson text).
- Optional: restore overlap word counts on the RapidOCR path; finer same-page image split everywhere.
- v2: disk cache, skip re-upload if PDF hash unchanged, background extract, no free Cloudflare tunnel for production.

These were treated as known limits, not silent failures.

8. Skills demonstrated

- Systems design: split CPU extract from GPU/LLM chat; split Azure resources by capability.
- Performance debugging: phase timers, worker pools, encoding deferral, header sampling.
- Production-minded ML: small local model versus cloud chat versus no model when regex is enough.
- Computer vision and PDF: soft masks, colour spaces, baked-in diagram text versus photographs.
- Frontend UX: streaming, polling races, layout experiments with revert, accessibility of source versus explanation.
- Operations: Kaggle datasets, tunnels, secrets hygiene (never log token values), Foundry deployment names versus model names.

9. Conclusion

The internship was spent building a real textbook pipeline, not a demo on three pages. The important technical lesson was to measure the bottleneck, isolate credentials and providers, and replace an unstable language-model step with a deterministic table of contents so the rest of the product (explain, ask, listen, images) could be finished and tested on a 1,299-page book.

I am comfortable presenting the week log, the challenge table, and a live demo: upload Class 9 Science (or a page range), open a topic, stream an explanation, ask a grounded question, Listen, and Generate HD only on a decorative photo.

Appendix

Supporting material for the Chapterwise internship at ADS Softek. This appendix is not a second narrative of the weeks above; it records dates, numbers, APIs, and terms used in the report.

Appendix A — Working days (from project chats)

Date	Week	Nature of work
Thu 23 Jul 2026	1	Steps 1–3: upload/extract, topic builder, React tree; 500 MB cap; Azure env; CPU-parallel extract

Fri 24 Jul 2026	1	Kaggle remote extract; chunked upload; Cloudflare 524; Azure chat missing
Sat 25 Jul 2026	1	Clarified missing chat deployment; re-checked TTS/image endpoints
Mon 27 Jul 2026	1	TTS/image smoke tests; topic view shell; upload loading UI
Tue 28 Jul 2026	1	Azure audit; /debug/azure; upload timing
Thu 30 Jul 2026	2	Kaggle Llama as text provider
Mon 3 Aug 2026	2	Llama explanations; JSON repair; Hugging Face gated access
Tue 4 Aug 2026	2	RapidOCR decision
Mon 10 Aug 2026	3	RapidOCR path; extract timing; header-cache; Llama OOM caps
Tue 11 Aug 2026	3	Kaggle/HF secrets; async chat (avoid 524)
Wed 12 Aug 2026	3	Chat reliability; regex TOC (Plan B)
Thu 13 Aug 2026	4	Regex confirmed live; fail-fast dead Kaggle; separate Azure chat resource
Fri 14 Aug 2026	4	Full-book extract; on-demand images; grounded Ask; full-book TOC
Mon 17 Aug 2026	4	SMask fix; image classify/HD; markdown; baked-text OCR gate
Wed 19 Aug 2026	4	Ask UX experiments; FAB + overlay drawer
Thu 20 Aug 2026	5	SSE streaming; TTS voices; collapsible source; images.edit
Fri 21 Aug 2026	5	Regression pass
Wed 26 Aug 2026	5	Heading_y / image_y matching on RapidOCR path
Wed 2 Sep 2026	6	Last working day: line-start Chapter N regex; DocLayout-YOLO experiment

No implementation chats exist after 2 September 2026.

Appendix B — Key measurements

Metric	Result
--------	--------

Reference PDF	NCERT Class 9 Science, ~301 MB, 1,299 pages
Partial extract (pages 1–10), after header-cache	~6–8 seconds (was ~99 s when headers scanned the whole book)
Full-book extract, first timed run	~287 s, peak ~2.4 GB private memory
Full-book extract, after defer-b64 + single RapidOCR init	~209 s
Regex TOC, pages 1–8	~21 topics in ~0.01 s
Regex TOC, full book	223 topics / 214 headings / 0.16 s
Dead-Kaggle topic open (after fail-fast)	Source + explanation failed in ~3 s (was up to 600 s)
Default TTS voice	shimmer (10 voices confirmed on the deployment)

Appendix C — Backend API surface (as of last working day)

Method	Path	Purpose
GET	/health	Health, providers, LLM/Kaggle status
POST	/upload_pdf	Save PDF and extract pages (optional page_start / page_end)
POST	/build_topics	Ordered topic tree (regex default)
GET	/topic/{id}	Topic metadata, source, images (no LLM call)
GET	/topic/{id}/explanation/stream	SSE explanation
POST	/topic/{id}/images/{image_id}/generate	On-demand HD edit for decorative images only
POST	/ask	SSE grounded Q&A for the open topic
POST	/tts	MP3 narration
GET	/debug/text	Chat provider smoke test
GET	/debug/azure	TTS and image smoke test
GET	/debug/extraction	Per-page extract summary
GET	/debug/topics	Built tree

Frontend talks to <http://127.0.0.1:8766> from `frontend/src/config/api.js` only.

Appendix D — Typical laptop defaults (no secrets)

Setting	Value used in development
---------	---------------------------

Extract	Local (EXTRACT_PROVIDER=local)
Text extract	RapidOCR; digital pages may use native PDF text
Topic builder	Regex (LLM path optional)
Chat	Azure gpt-4.1-mini on a dedicated chat resource
TTS / images	Separate Azure services resource
Image bytes	Deferred until a topic is opened
Decorative HD	On-demand images.edit; never auto on topic open

Appendix E — Glossary

Term	Meaning in this project
Topic tree	Ordered chapter → section → subsection list that follows the textbook
Regex TOC	Heading scan that builds the tree without an LLM
Grounded Q&A	Answers only from the open topic's explanation and source
Informational image	Diagram/formula/labelled figure; Lanczos only; never gpt-image-2
Decorative image	Photo/illustration eligible for on-demand HD edit
SMask	PDF soft mask; if decoded wrongly, artwork can look solid black
SSE	Server-Sent Events; incremental explanation and Ask text
Fail soft	Show source and a status instead of hanging the UI

Appendix F — Sources for this report

- Chapterwise project chat transcripts (23 July 2026 – 2 September 2026)
- Git commit history on the Chapterwise repository
- Project changelog / session notes used during development

Secrets, API keys, and tunnel URLs from those chats are omitted.

End of report